Chapter 9 Strings and Text I/O

Constructing Strings

```
String newString = new String(stringLiteral);

String message = new String("Welcome to Java");
```

Since strings are used frequently, Java provides a shorthand initializer for creating a string:

```
String message = "Welcome to Java";
```

String Comparisons

- **equals**

```
String s1 = new String("Welcome“);
String s2 = "welcome";

if (s1.equals(s2)){
  // s1 and s2 have the same contents
}

if (s1 == s2) {
  // s1 and s2 have the same reference
}
```

String Comparisons, cont.

- **compareTo()**

```
String s1 = new String("Welcome“);
String s2 = "welcome";

if (s1.compareTo(s2) > 0) {
  // s1 is greater than s2
}
else if (s1.compareTo(s2) == 0) {
  // s1 and s2 have the same contents
}
else
   // s1 is less than s2
```

Finding String Length

Finding string length using the `length()` method:

```
message = "Welcome";
message.length()
```

returns `7`

Retrieving Individual Characters in a String

- Do not use `message[0]`
- Use `message.charAt(index)`
- Index starts from `0`

Indices	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
message	W	e	l	c	o	m	e		t	o		J	a	v	a

message.charAt(0) message.length() is 15 message.charAt(14)

String Concatenation

```
String s3 = s1.concat(s2);
```

String s3 = s1 + s2;

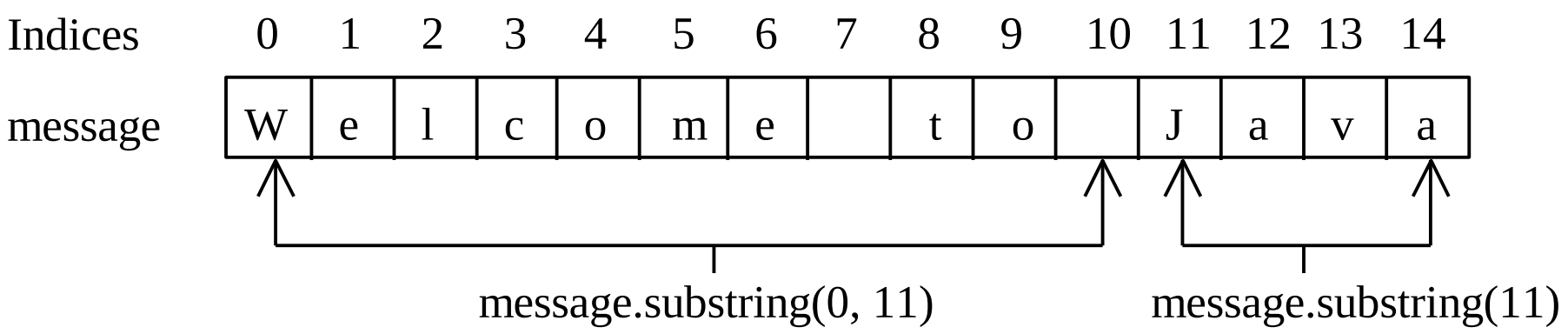
s1 + s2 + s3 + s4 + s5 same as

(((s1.concat(s2)).concat(s3)).concat(s4)).concat(s5);

Extracting Substrings

You can extract a single character from a string using the charAt method. You can also extract a substring from a string using the substring method in the String class.

```
String s1 = "Welcome to Java";
String s2 = s1.substring(0, 11) + "HTML";
```



Converting, Replacing, and Splitting Strings

java.lang.String	
+toLowerCase(): String	Returns a new string with all characters converted to lowercase.
+toUpperCase(): String	Returns a new string with all characters converted to uppercase.
+trim(): String	Returns a new string with blank characters trimmed on both sides.
+replace(oldChar: char, newChar: char): String	Returns a new string that replaces all matching character in this string with the new character.
+replaceFirst(oldString: String, newString: String): String	Returns a new string that replaces the first matching substring in this string with the new substring.
+replaceAll(oldString: String, newString: String): String	Returns a new string that replace all matching substrings in this string with the new substring.
+split(delimiter: String): String[]	Returns an array of strings consisting of the substrings split by the delimiter.

Examples

"Welcome".toLowerCase() returns a new string, welcome.

"Welcome".toUpperCase() returns a new string, WELCOME.

" Welcome ".trim() returns a new string, Welcome.

"Welcome".replace('e', 'A') returns a new string, WAlcomA.

"Welcome".replaceFirst("e", "AB") returns a new string, WABlcome.

"Welcome".replace("e", "AB") returns a new string, WABlcomAB.

"Welcome".replace("el", "AB") returns a new string, WABlcome.

Splitting a String

```
String[] tokens = "Java#HTML#Perl".split("#", 0);
for (int i = 0; i < tokens.length; i++)
  System.out.print(tokens[i] + " ");
```

displays

Java HTML Perl

Finding a Character or a Substring in a String

java.lang.String	
+indexOf(ch: char): int	Returns the index of the first occurrence of ch in the string. Returns -1 if not matched.
+indexOf(ch: char, fromIndex: int): int	Returns the index of the first occurrence of ch after fromIndex in the string. Returns -1 if not matched.
+indexOf(s: String): int	Returns the index of the first occurrence of string s in this string. Returns -1 if not matched.
+indexOf(s: String, fromIndex: int): int	Returns the index of the first occurrence of string s in this string after fromIndex. Returns -1 if not matched.
+lastIndexOf(ch: int): int	Returns the index of the last occurrence of ch in the string. Returns -1 if not matched.
+lastIndexOf(ch: int, fromIndex: int): int	Returns the index of the last occurrence of ch before fromIndex in this string. Returns -1 if not matched.
+lastIndexOf(s: String): int	Returns the index of the last occurrence of string s. Returns -1 if not matched.
+lastIndexOf(s: String, fromIndex: int): int	Returns the index of the last occurrence of string s before fromIndex. Returns -1 if not matched.

Finding a Character or a Substring in a String

`"Welcome to Java".indexOf('W')` returns 0.

`"Welcome to Java".indexOf('x')` returns -1.

`"Welcome to Java".indexOf('o', 5)` returns 9.

`"Welcome to Java".indexOf("come")` returns 3.

`"Welcome to Java".indexOf("Java", 5)` returns 11.

`"Welcome to Java".indexOf("java", 5)` returns -1.

`"Welcome to Java".lastIndexOf('a')` returns 14.

Convert Character and Numbers to Strings

The String class provides several static valueOf methods for converting a character, an array of characters, and numeric values to strings. These methods have the same name valueOf with different argument types char, char[], double, long, int, and float. For example, to convert a double value to a string, use String.valueOf(5.44). The return value is string consists of characters ‘5’, ‘.’, ‘4’, and ‘4’.

Problem: Finding Palindromes

- Objective: Checking whether a string is a palindrome: a string that reads the same forward and backward.

CheckPalindrome

Run

The `Character` Class

java.lang.Character	
+Character(value: char)	Constructs a character object with char value
+charValue(): char	Returns the char value from this object
+equals(anotherCharacter: Character): boolean	Returns true if this character equals to another
+isDigit(ch: char): boolean	Returns true if the specified character is a digit
+isLetter(ch: char): boolean	Returns true if the specified character is a letter
+isLetterOrDigit(ch: char): boolean	Returns true if the character is a letter or a digit
+isLowerCase(ch: char): boolean	Returns true if the character is a lowercase letter
+isUpperCase(ch: char): boolean	Returns true if the character is an uppercase letter
+toLowerCase(ch: char): char	Returns the lowercase of the specified character
+toUpperCase(ch: char): char	Returns the uppercase of the specified character

Examples

```
Character x = new Character('b');
System.out.println(Character.isDigit(x));
//returns false

 char x = 'b';
 System.out.println(Character.isDigit(x));
//returns false
```

`StringBuilder` and `StringBuffer`

The `StringBuilder`/`StringBuffer` class is an alternative to the `String` class. In general, a StringBuilder/StringBuffer can be used wherever a string is used. StringBuilder/StringBuffer is more flexible than String. You can add, insert, or append new contents into a string buffer, whereas the value of a String object is fixed once the string is created.

Modifying Strings in the Builder

java.lang.StringBuilder	
+append(data: char[]): StringBuilder	Appends a char array into this string builder.
+append(data: char[], offset: int, len: int): StringBuilder	Appends a subarray in data into this string builder.
+append(v: *aPrimitiveType*): StringBuilder	Appends a primitive type value as a string to this builder.
+append(s: String): StringBuilder	Appends a string to this string builder.
+delete(startIndex: int, endIndex: int): StringBuilder	Deletes characters from startIndex to endIndex.
+deleteCharAt(index: int): StringBuilder	Deletes a character at the specified index.
+insert(index: int, data: char[], offset: int, len: int): StringBuilder	Inserts a subarray of the data in the array to the builder at the specified index.
+insert(offset: int, data: char[]): StringBuilder	Inserts data into this builder at the position offset.
+insert(offset: int, b: *aPrimitiveType*): StringBuilder	Inserts a value converted to a string into this builder.
+insert(offset: int, s: String): StringBuilder	Inserts a string into this builder at the position offset.
+replace(startIndex: int, endIndex: int, s: String): StringBuilder	Replaces the characters in this builder from startIndex to endIndex with the specified string.
+reverse(): StringBuilder	Reverses the characters in the builder.
+setCharAt(index: int, ch: char): void	Sets a new character at the specified index in this builder.

Examples

```
StringBuilder sb=new StringBuilder("Hello ");
sb.append("Java");
System.out.println(sb);
sb.insert(4, "Sam");
System.out.println(sb);
sb.delete(1,5);
System.out.println(sb);
sb.deleteCharAt(5);
System.out.println(sb);
sb.replace(1,4,"ello");
System.out.println(sb);
sb.setCharAt(4, 'a');
System.out.println(sb);
sb.reverse();
System.out.println(sb);
```

Hello Java
HellSamo Java
Hamo Java
Hamo ava
Hello ava
Hella ava
ava alleH

The File Class

The File class is intended to provide an abstraction that deals with most of the machine-dependent complexities of files and path names in a machine-independent fashion. The filename is a string. The File class is a wrapper class for the file name and its directory path.

Obtaining file properties and manipulating file

java.io.File	
+File(pathname: String)	Creates a File object for the specified pathname. The pathname may be a directory or a file.
+File(parent: String, child: String)	Creates a File object for the child under the directory parent. child may be a filename or a subdirectory.
+File(parent: File, child: String)	Creates a File object for the child under the directory parent. parent is a File object. In the preceding constructor, the parent is a string.
+exists(): boolean	Returns true if the file or the directory represented by the File object exists.
+canRead(): boolean	Returns true if the file represented by the File object exists and can be read.
+canWrite(): boolean	Returns true if the file represented by the File object exists and can be written.
+isDirectory(): boolean	Returns true if the File object represents a directory.
+isFile(): boolean	Returns true if the File object represents a file.
+isAbsolute(): boolean	Returns true if the File object is created using an absolute path name.
+isHidden(): boolean	Returns true if the file represented in the File object is hidden. The exact definition of *hidden* is system-dependent. On Windows, you can mark a file hidden in the File Properties dialog box. On Unix systems, a file is hidden if its name begins with a period character '.'.
+getAbsolutePath(): String	Returns the complete absolute file or directory name represented by the File object.
+getCanonicalPath(): String	Returns the same as getAbsolutePath() except that it removes redundant names, such as "." and "..", from the pathname, resolves symbolic links (on Unix platforms), and converts drive letters to standard uppercase (on Win32 platforms).
+getName(): String	Returns the last name of the complete directory and file name represented by the File object. For example, new File("c:\\book\\test.dat").getName() returns test.dat.
+getPath(): String	Returns the complete directory and file name represented by the File object. For example, new File("c:\\book\\test.dat").getPath() returns c:\book\test.dat.
+getParent(): String	Returns the complete parent directory of the current directory or the file represented by the File object. For example, new File("c:\\book\\test.dat").getParent() returns c:\book.
+lastModified(): long	Returns the time that the file was last modified.
+delete(): boolean	Deletes this file. The method returns true if the deletion succeeds.
+renameTo(dest: File): boolean	Renames this file. The method returns true if the operation succeeds.

Problem: Explore File Properties

```
public static void main(String[] args) {

    java.io.File file = new java.io.File("c:\\test\\sample.txt");
    System.out.println("Does it exist? " + file.exists());
    System.out.println("The file has " + file.length() + " bytes");
    System.out.println("Can it be read? " + file.canRead());
    System.out.println("Can it be written? " + file.canWrite());
    System.out.println("Is it a directory? " + file.isDirectory());
    System.out.println("Is it a file? " + file.isFile());
    System.out.println("Is it absolute? " + file.isAbsolute());
    System.out.println("Is it hidden? " + file.isHidden());
    System.out.println("Absolute path is " + file.getAbsolutePath());
    System.out.println("Last modified on " + new
java.util.Date(file.lastModified()));
    }
```

Command Prompt

```
C:\book>java TestFileClass
Does it exist? true
Can it be read? true
Can it be written? true
Is it a directory? false
Is it a file? true
Is it absolute? false
Is it hidden? false
What is its absolute path? C:\book\.\image\us.gif
What is its canonical path? C:\book\image\us.gif
What is its name? us.gif
What is its path? .\image\us.gif
When was it last modified? Sat May 08 14:00:34 EDT 1999
What is the path separator? ;
What is the name separator? \

C:\book>
```

TestFileClass

Text I/O

A File object encapsulates the properties of a file or a path, but does not contain the methods for reading/writing data from/to a file. In order to perform I/O, you need to create objects using appropriate Java I/O classes. The objects contain the methods for reading/writing data from/to a file. This section introduces how to read/write strings and numeric values from/to a text file using the Scanner and PrintWriter classes.

Writing Data Using PrintWriter

```
import java.io.*;
public class JavaApplication6 {
    public static void main(String[] args) throws
FileNotFoundException {
    File file = new File("c:\\test\\scores.txt");
    if (file.exists()) {
      System.out.println("File already exists");
      System.exit(0);
    }
    // Create a file
    PrintWriter output = new PrintWriter(file);
    // Write formatted output to the file
    output.print("John T Smith ");
    output.println(90);
    output.print("Eric K Jones ");
    output.println(85);
    // Close the file
    output.close();
    }
}
```

WriteData

Writing Data Using FileWriter

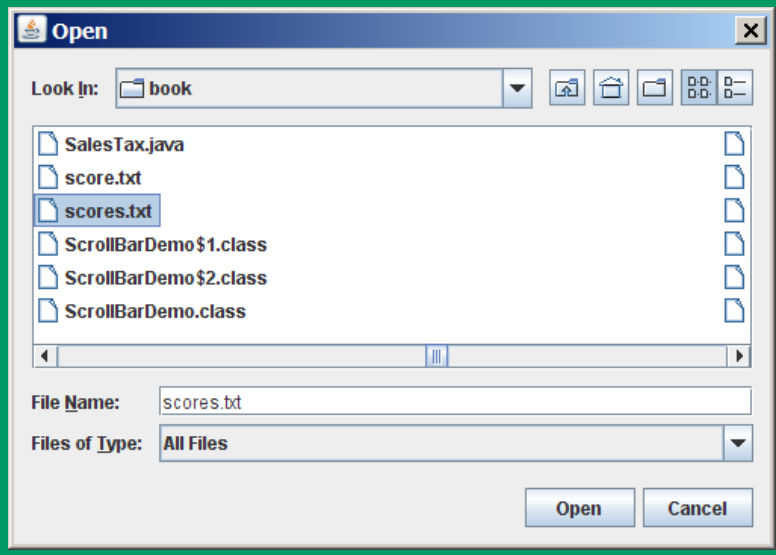
```
import java.io.*;
public class JavaApplication6 {
    public static void main(String[] args) throws IOException {
    String dosya = "c:\\test\\scores.txt";
        // Create a file
        FileWriter output = new FileWriter(dosya,true);
        // Write formatted output to the file
        output.write("John T Smith ");
        output.write("90\r\n");
        output.write("Eric K Jones ");
        output.write("85\r\n");
        // Close the file
        output.close();
  }
}
```

WriteData

Reading Data Using Scanner

```
import java.io.*;
import java.util.Scanner;
public class JavaApplication6 {
    public static void main(String[] args) throws FileNotFoundException {
    File file = new File("c:\\test\\scores.txt");
    // Create a Scanner for the file
    Scanner input = new Scanner(file);
    // Read data from a file
    while (input.hasNext()) {
      String firstName = input.next();
      String mi = input.next();
      String lastName = input.next();
      int score = input.nextInt();
      System.out.println(
        firstName + " " + mi + " " + lastName + " " + score);
    }
    // Close the file
    input.close();
    }
}
```

ReadData

(GUI) File Dialogs

```
import java.io.*;
import java.util.Scanner;
import javax.swing.JFileChooser;
import javax.swing.JOptionPane;
public class JavaApplication6 {
    public static void main(String[] args) throws FileNotFoundException {
        JFileChooser fileChooser = new JFileChooser();
        if (fileChooser.showOpenDialog(null) == JFileChooser.APPROVE_OPTION) {
            // Get the selected file
            File file = fileChooser.getSelectedFile();
            // Create a Scanner for the file
            Scanner input = new Scanner(file);
            // Read text from the file
            String text = "";
            while (input.hasNext()) {
                text += input.nextLine() + "\n";
            }
            JOptionPane.showMessageDialog(null, text);
            // Close the file
            input.close();
        } else {
            System.out.println("No file selected");
        }
    }
}
```